



佛山大学
FOSHAN UNIVERSITY



计算机与人工智能学院
School of Computer Science and Artificial Intelligence

2025

NLP—深度学习

Natural Language Processing — Deep Learning

任课教师：陈荟慧

目录 CONTENTS



01

循环神经网络

02

长短时记忆神经网络

03

Seq2Seq问答机器人



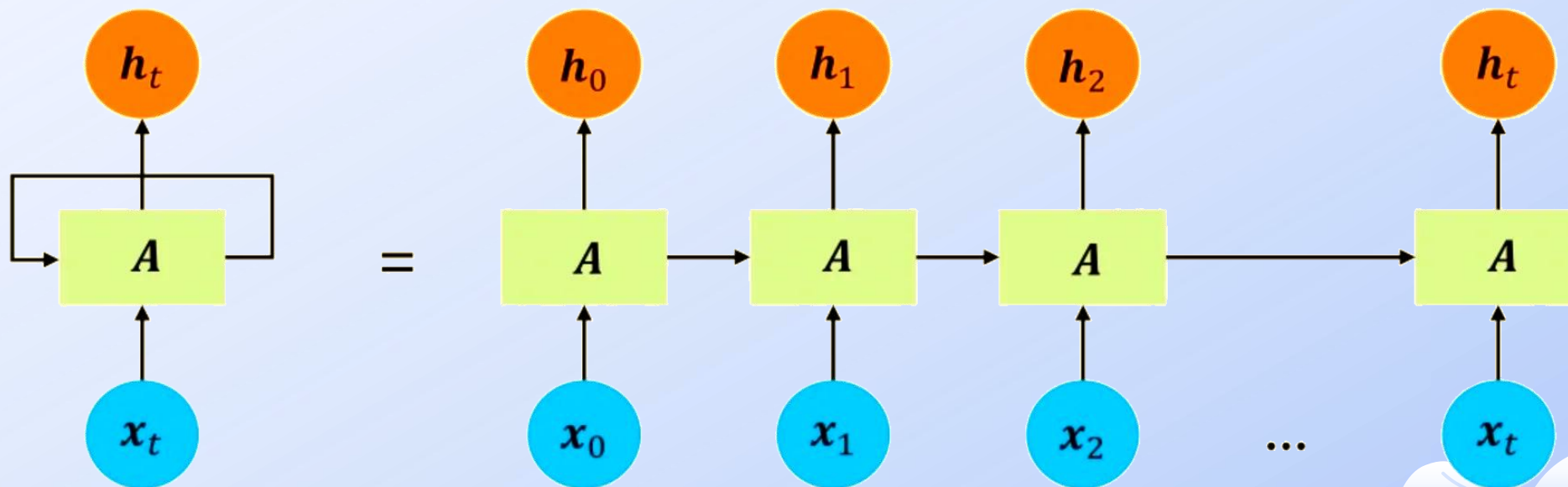
循环神经网络概述

- 循环神经网络 (Recurrent Neural Network, RNN) 是一种特殊的神经网络结构, 根据“**人的认知是基于过往的经验和记忆**”这一观点提出。
- RNN的研究始于二十世纪80-90年代, 常见的RNN有双向循环神经网络 (Bidirectional RNN, Bi-RNN) 和长短期记忆网络 (Long Short-Term Memory networks, LSTM) 。
- RNN适用于处理视频、语音、文本等与时序相关的问题。常见的应用领域有文本生成、语言模型、图像处理、机器翻译、语音识别等。



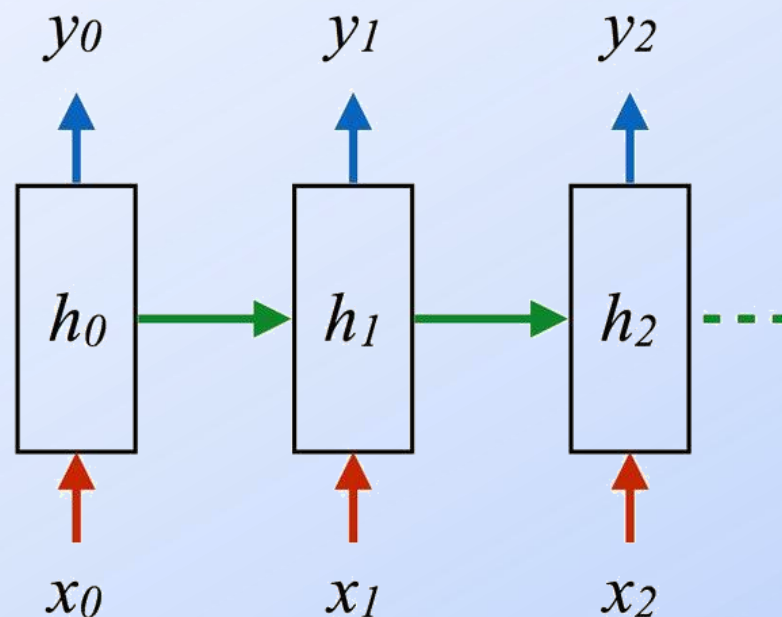
循环神经网络概述

- RNN以序列数据为输入，在序列的演进方向进行递归，并且所有节点按链式连接的递归神经网络。
- RNN中，神经元不但可以接收其他神经元的信息，还可以接收自身的信息，形成具有环路的网络结构。



循环神经网络(Recurrent NN, RNN)

- 对文本进行逐词分析，将已有的文本信息存储在固定大小的隐藏层。
- 时间复杂度： $O(n)$
- 缺点：不公平性，后出现的单词比前出现的单词重要性更高。因此对于整个文本进行分类时会降低有效性。因为文本中重要的单词会在任何地方出现。



1	循环神经网络概述
2	RNN结构
3	深度学习工具
4	基于LSTM的文本分类与情感分析
5	基于Seq2Seq的机器翻译



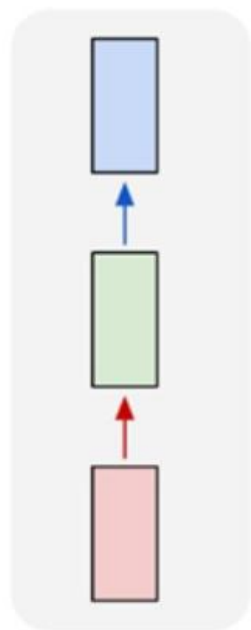
- h_t —在 t 时刻RNN对应的输出数据
- U —输入相关的权重矩阵
- W —隐藏状态相关的权重矩阵
- V —输出相关的权重矩阵

$$s_t = f(UX_t + Ws_{t-1})$$

$$h_t = \text{softmax}(Vs_t)$$

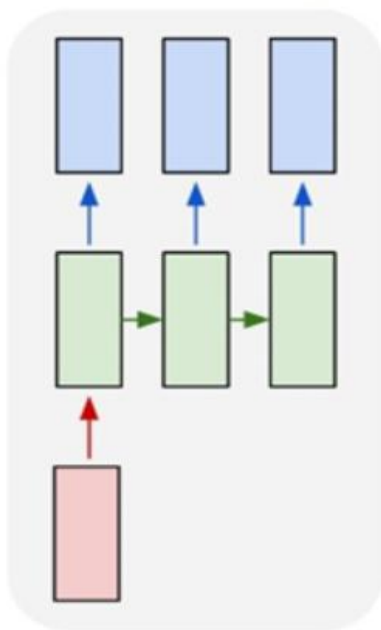
- X_t —在 t 时刻的输入数据
- s_t —在 t 时刻RNN的隐藏状态 (hidden state)

one to one



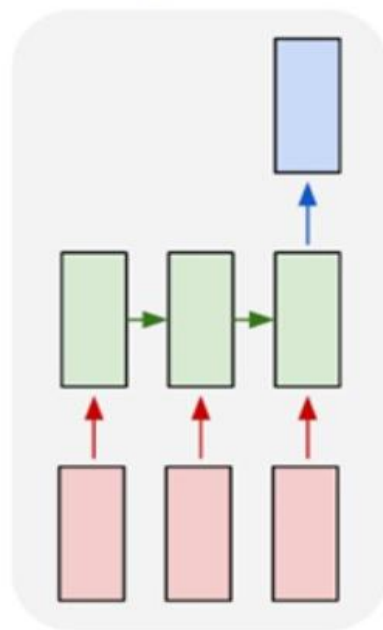
MLP

one to many



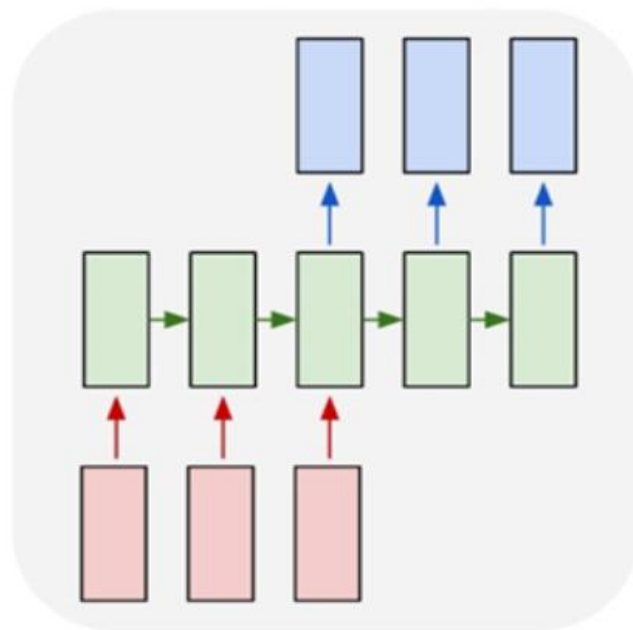
图像描述

many to one



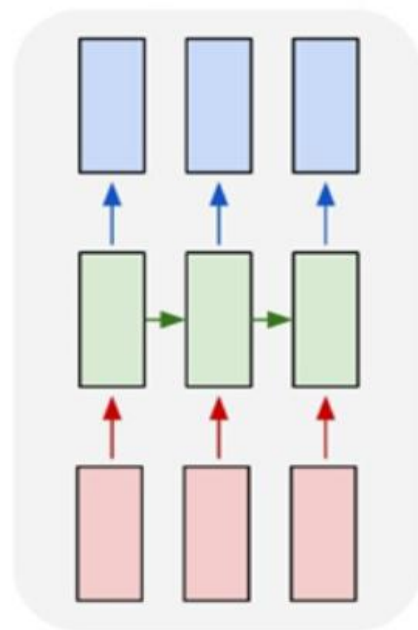
情感分类

many to many



机器翻译

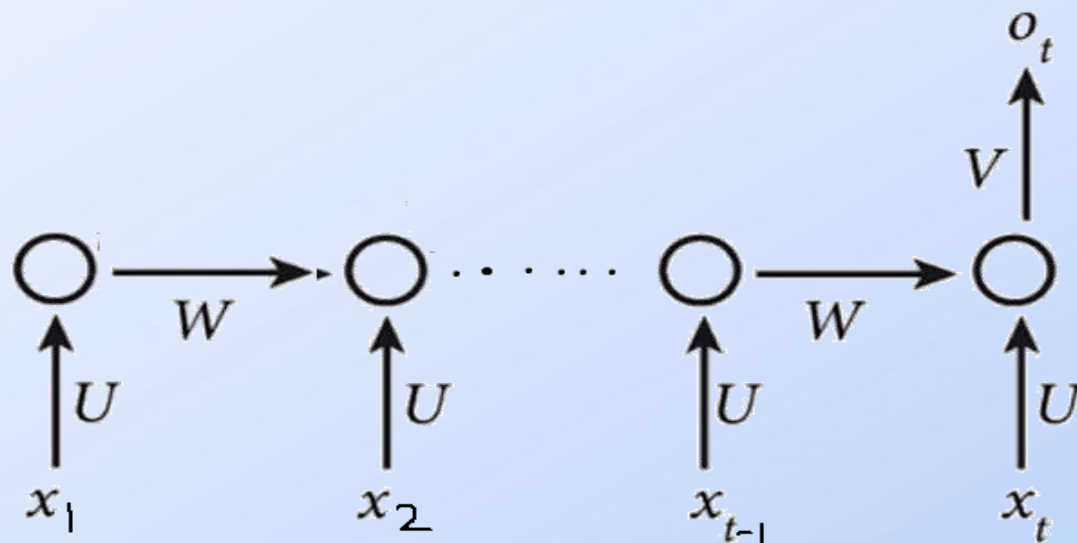
many to many



逐帧视频分

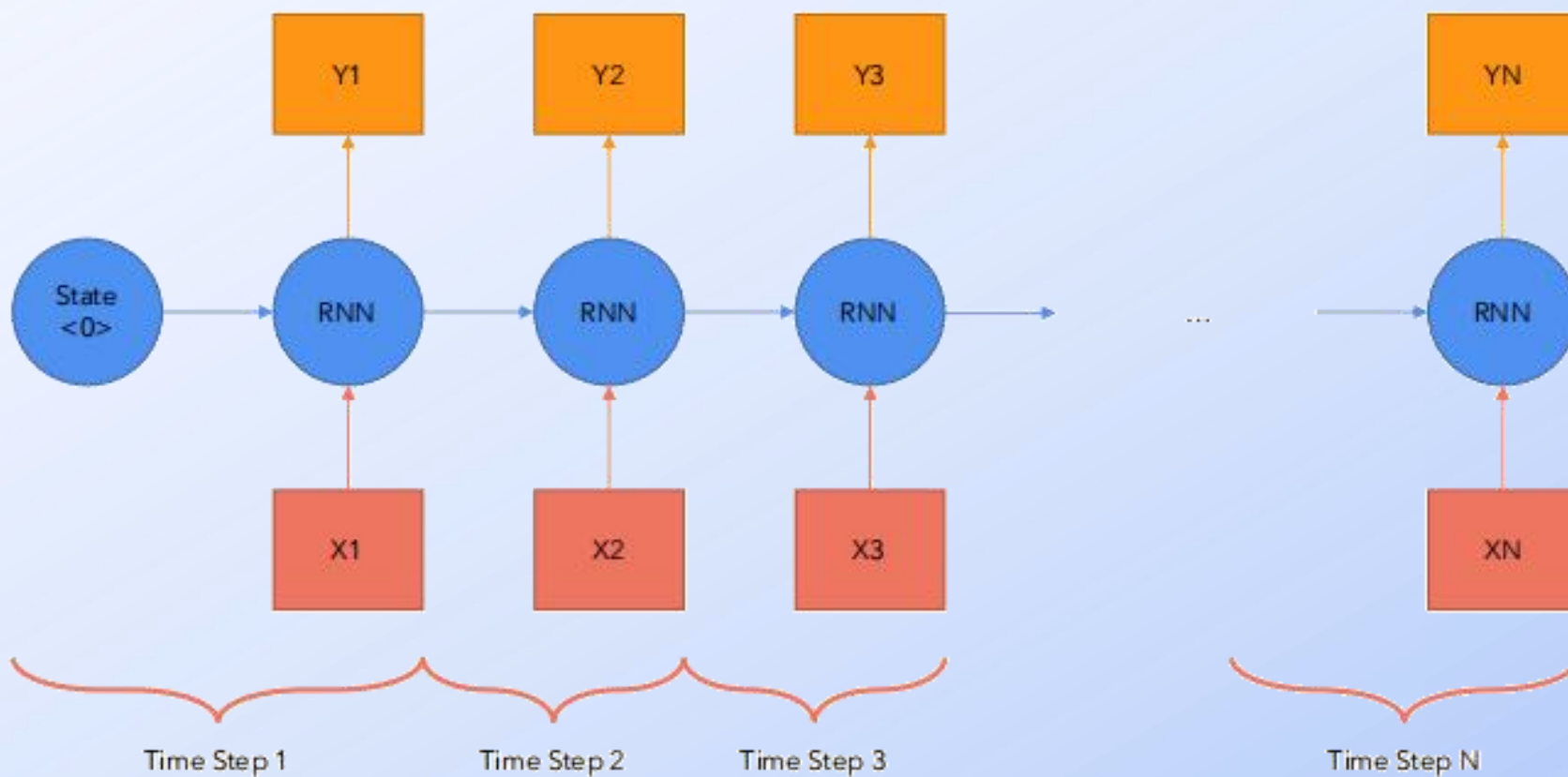
多对一结构

- RNN的多对一结构是指输入是一个序列，输出是一个单独的值而不是序列，如图所示。这种结构通常用于处理序列分类问题。如输入一段文字判别它所属的类别，输入一个句子判断其情感倾向等。
- $x_1, x_2, \dots, x_n$ 为输入， o 为输出。结构图中圆圈或方块表示的是向量。
- 不同步骤中的网络连接的权重矩阵 U 、 W 、 V 都是一样的，即每个步骤的参数都是共享的。



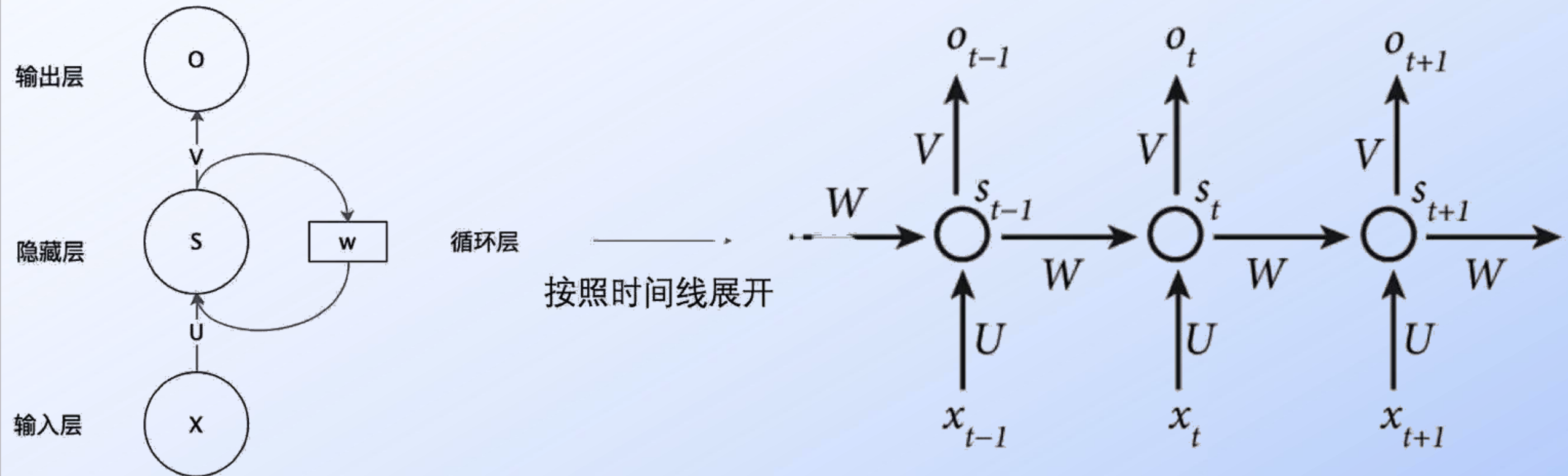
等长的多对多结构

- 等长的多对多RNN结构的输入和输出序列是等长的，这种结构是RNN的经典结构，可以用于生成文章，诗歌，甚至是代码。等长的多对多RNN结构如图所示。

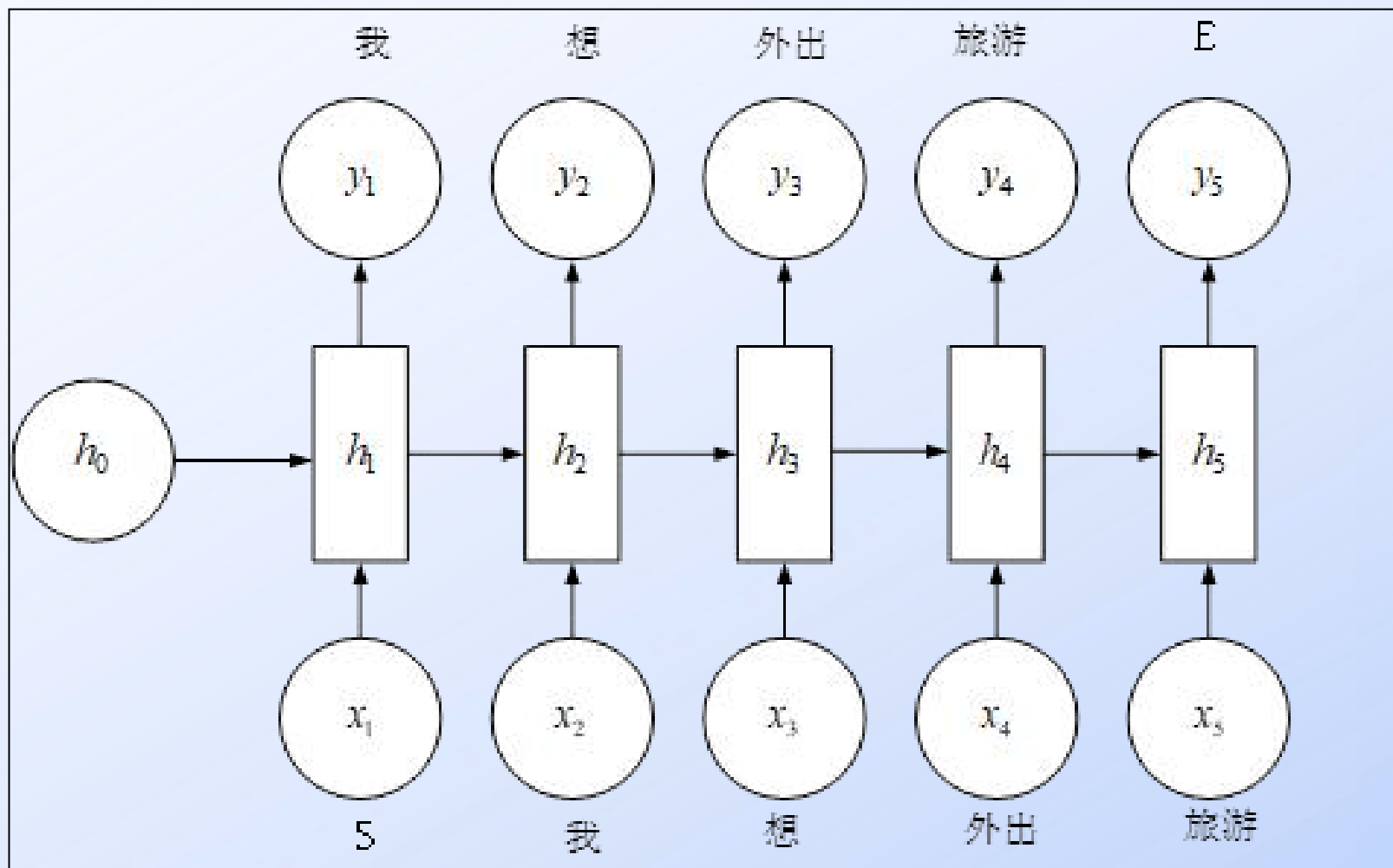


$$A = \pi r^2$$

按时间展开框架图

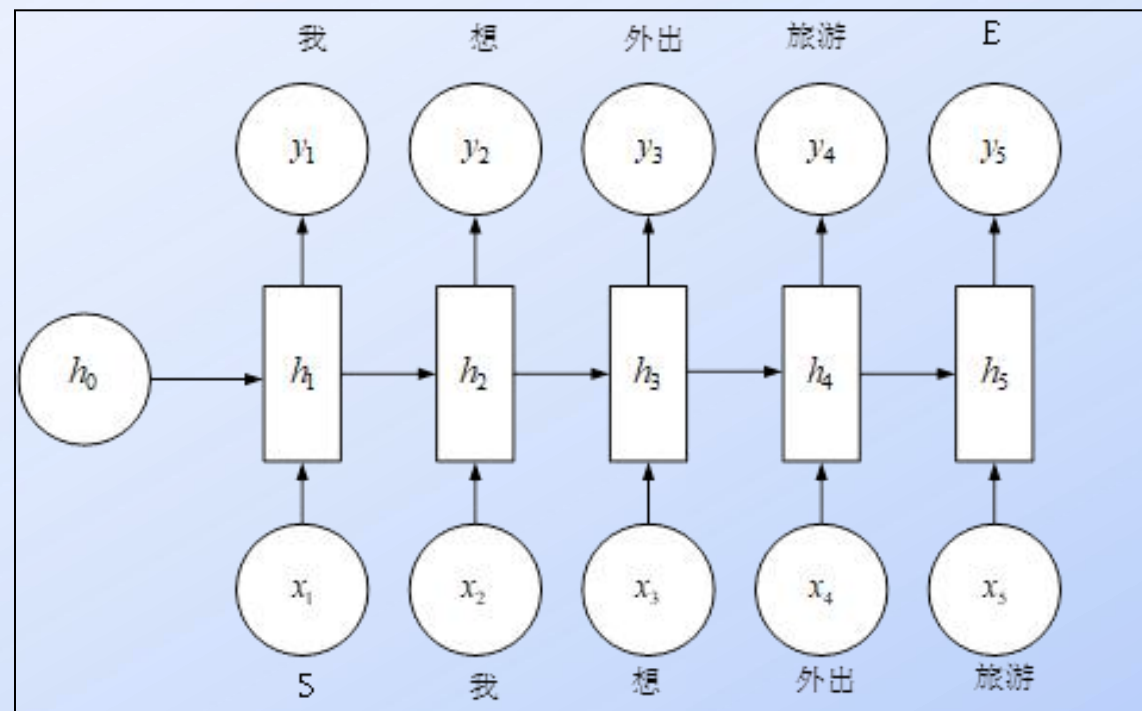


$$A = \pi r^2$$



等长的多对多结构-RNN语言模型

- 将等长的多对多RNN应用于语言模型时，需要将词依次输入到网络中。
- 每输入一个词，RNN将输出截止到目前为止下一个最可能的词。例如，当依次输入“我”“想”“外出”“旅游”时，RNN的输入、输出如图所示，其中S和E是两个特殊的词，分别表示序列的开始与结束。

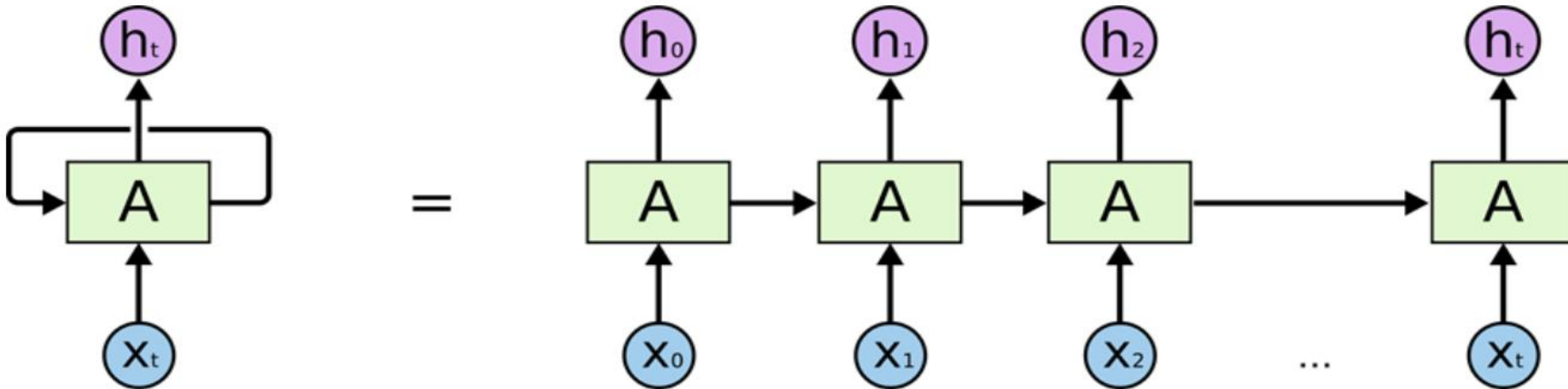
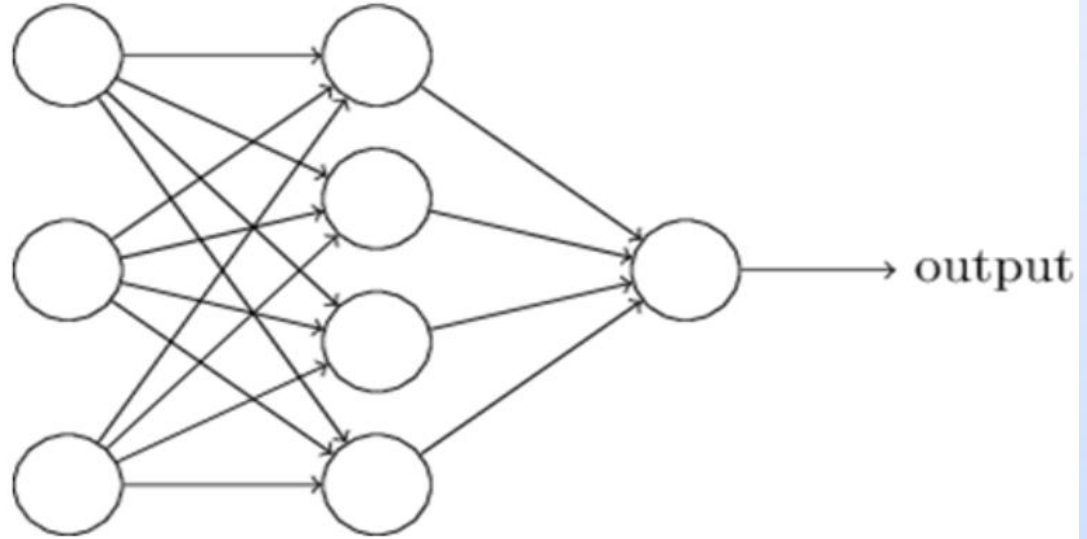
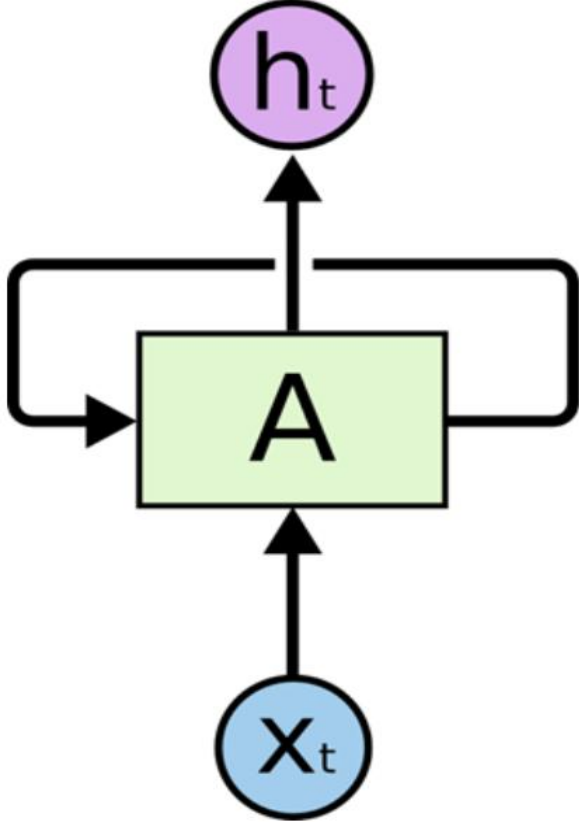




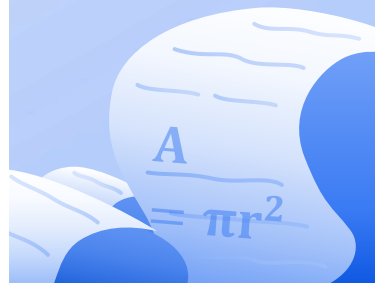
等长的多对多结构

- RNN语言模型可以描述为：
 - 给定词序列 $x_1, x_2, \dots, x_t$ ，通过计算式 $P(x_{t+1} = v_j | x_t, \dots, x_1)$ 预测序列下一个词 x_{t+1} 的概率。
- RNN网络只能接收数字向量，无法直接接收语言文本，因此**需要将词表达为向量的形式**。
- 为了将词向量化，需要执行下面两个步骤。
 - 建立一个包含所有词的词典，每个词在词典中具有唯一的编号。
 - 使用K维的one-hot向量表示词典中的每一个词，其中K为词典中包含词的个数。





- X_t - 在 t 时刻的输入数据
- s_t - 在 t 时刻RNN的隐藏状态 (hidden state)





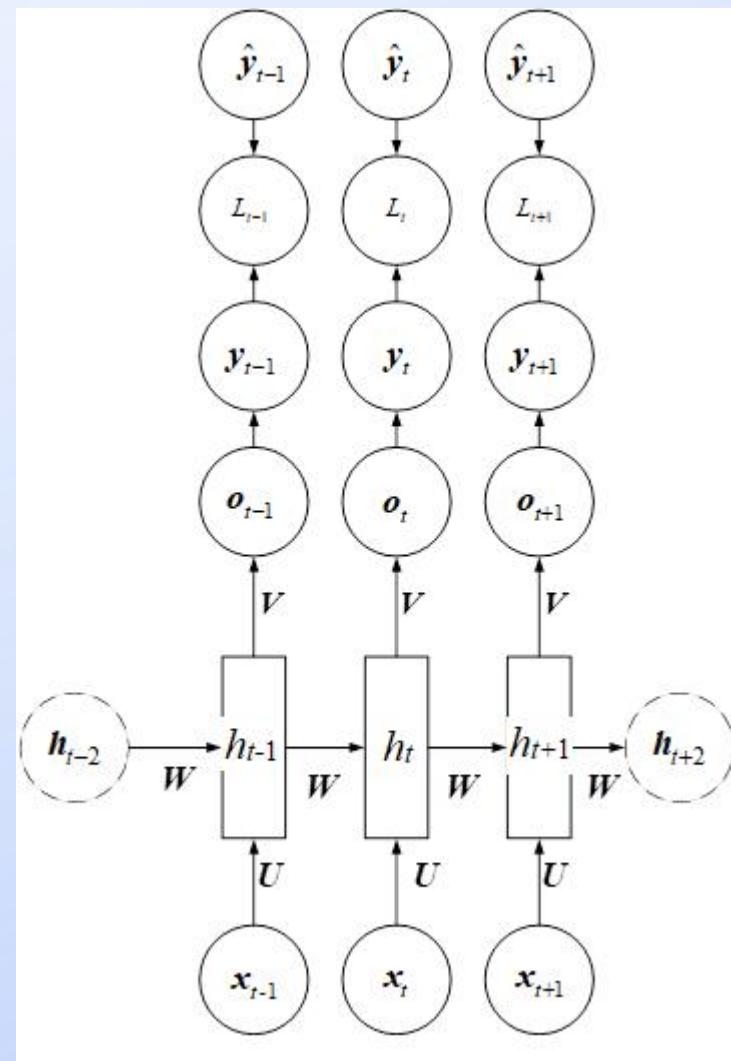
等长的多对多结构

- RNN参数和变量之间的关系描述如下八点。
- t 时刻输出的 x_t 为one-hot向量, $x_t \in R^{K \times 1}$ 。
- 输入层与隐藏层之间的共享权重矩阵为 $U \in R^{H \times K}$ 。
- 上一个时刻隐藏层与当前时刻隐藏层之间的共享权重矩阵为 $W \in R^{H \times H}$ 。
- 隐藏层与输出层之间的共享权重矩阵为 $V \in R^{K \times H}$ 。
- t 时刻隐藏层输入为 $s_t \in R^{H \times 1}$, 其中 $s_t = Wh_{t-1} + Ux_t + b$ 。
- t 时刻隐藏层的输出为 $h_t \in R^{K \times 1}$, $h_t = \sigma(s_t)$, σ 为sigmoid函数。
- 输出层的输入为 $o_t \in R^{K \times 1}$, 其中 $o_t = Vh_t + c$ 。
- t 时刻经过Softmax层的输出为 $y_t \in R^{K \times 1}$, $y_t = g(o_t)$ 。



等长的多对多结构-模型的训练

- 设 t 时刻经过Softmax层的输出为 y_t ，实际样本对应的词向量为 $\hat{y}_t \in R^{K \times 1}$ (one-hot向量)。RNN的前向传播过程如图所示。
- 使用交叉熵 $L_t(\theta) = -\sum_{i=1}^K \hat{y}_i^{(t)} \log(y_i^{(t)})$ 计算模型输出 y_t 与实际样本 $\hat{y}_t$ 的误差。
- 总的损失函数定义为所有交叉熵的均值 $L(\theta) = \frac{1}{T} \sum_{t=1}^T L_t(\theta)$ 。





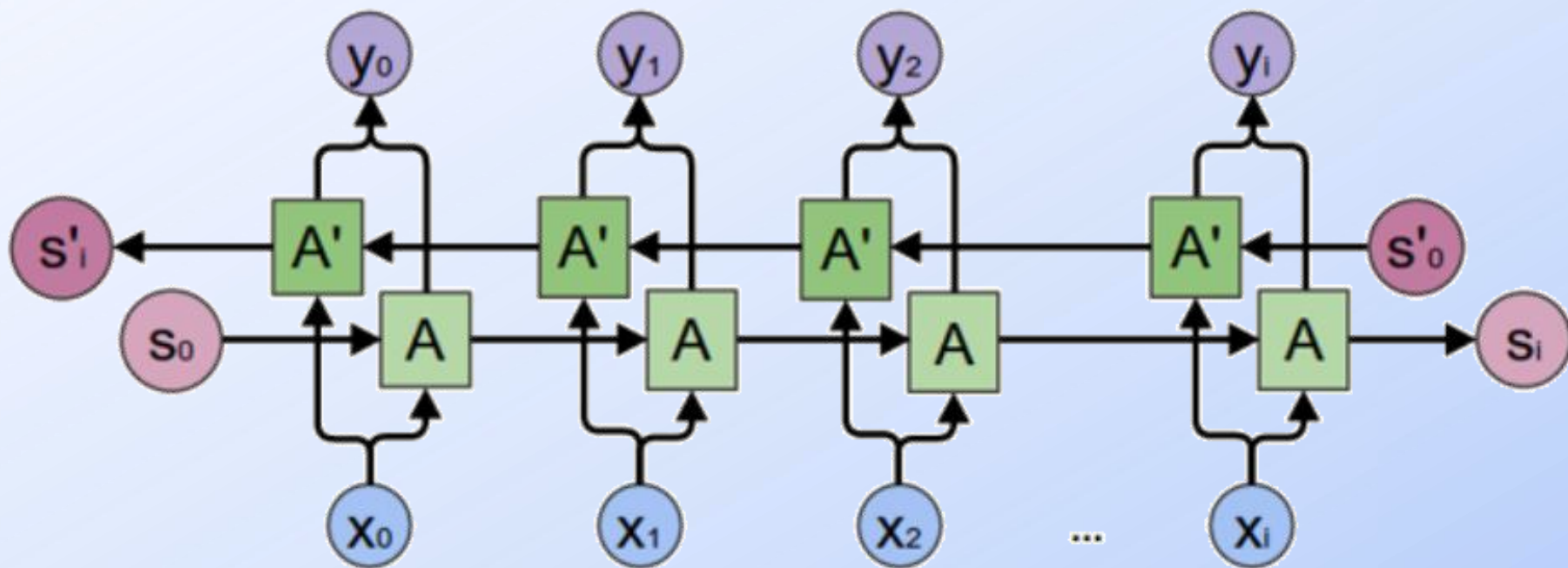
等长的多对多结构

- RNN中的权重矩阵U、W、V是未知参数，这几个参数需要通过模型训练获得。
- 模型训练的过程如下：
 - 加载一个大的文本语料库。
 - 将语料库放到RNN语言模型中，计算每个时刻t的输出。
 - 计算损失函数。
 - 反向传播用于RNN，计算函数的导数。
 - 由于在整个语料库上计算需要花费大量的时间，在实际中采用随机梯度下降法批量计算损失和梯度然后进行更新。
- 这些年研究者们又提出了多种复杂的RNN去改进RNN模型的性能，常见的有双向RNN（Bidirectional RNN）和LSTM模型。



双向RNN结构

- 双向RNN是由两个RNN上下叠加在一起组成，输出由两个RNN的隐藏层的状态决定。其结构如图所示。



$$A = \pi r^2$$



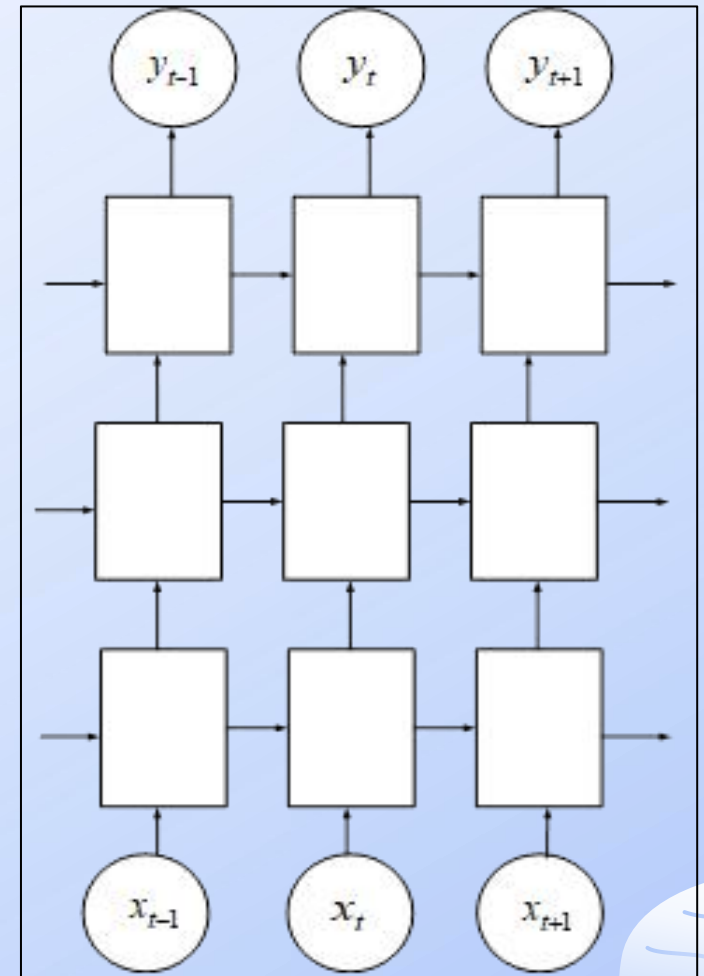
双向RNN结构

- 基本的RNN中只考虑预测词前面的词，并没有考虑该词后面的内容。这可能会错过一些重要的信息，使得预测的内容不够准确。
- 双向RNN不仅从前往后保留该词前面的词的信息，而且从后往前去保留该词后面的词的信息，然后基于这些信息进行预测该词。例如，如果预测一个语句中缺失的词语，那么需要根据上下文进行预测。



多层RNN结构

- RNN的另一种改进是多层RNN，结构和双向RNN类似，只是对于每一步的输入增加多层网络。
- 多层RNN有更强大的表达与学习能力，但复杂性也提高了，同时需要更多的训练数据。如一个三层RNN结构如图所示。
- 多层RNN每一层的参数和基本的RNN结构一样，参数共享，不同层的参数一般不共享。
- 与经典结构的RNN相比，多层RNN的泛化能力更强，但是训练的时间复杂度和空间复杂度也更高。



$$A = \pi r^2$$

目录 CONTENTS



01

循环神经网络

02

长短时记忆
神经网络

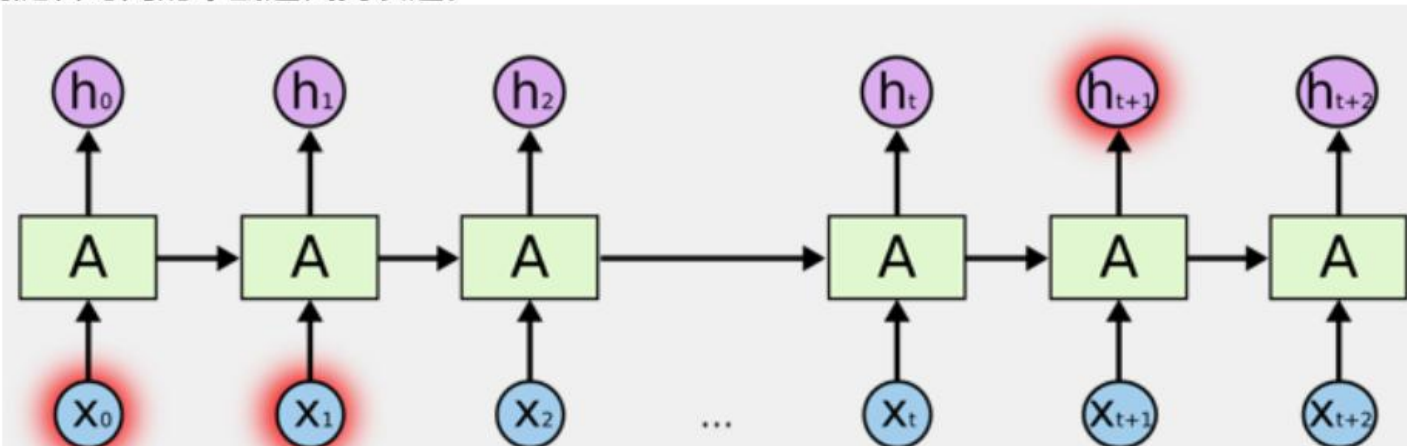
03

Seq2Seq问
答机器人

LSTM结构

- 基本的RNN参数通过Back propagation Through Time (BPTT) 算法实现，即将输出端的误差值反向传递，运用梯度下降法进行更新。
- 然而，在训练中，由于BPTT算法会带来梯度消失或梯度爆炸问题，因此，BPTT算法无法解决长时依赖问题。而RNN结构面对“长距离依赖”性能也会变差，很难学习到长距离的信息。

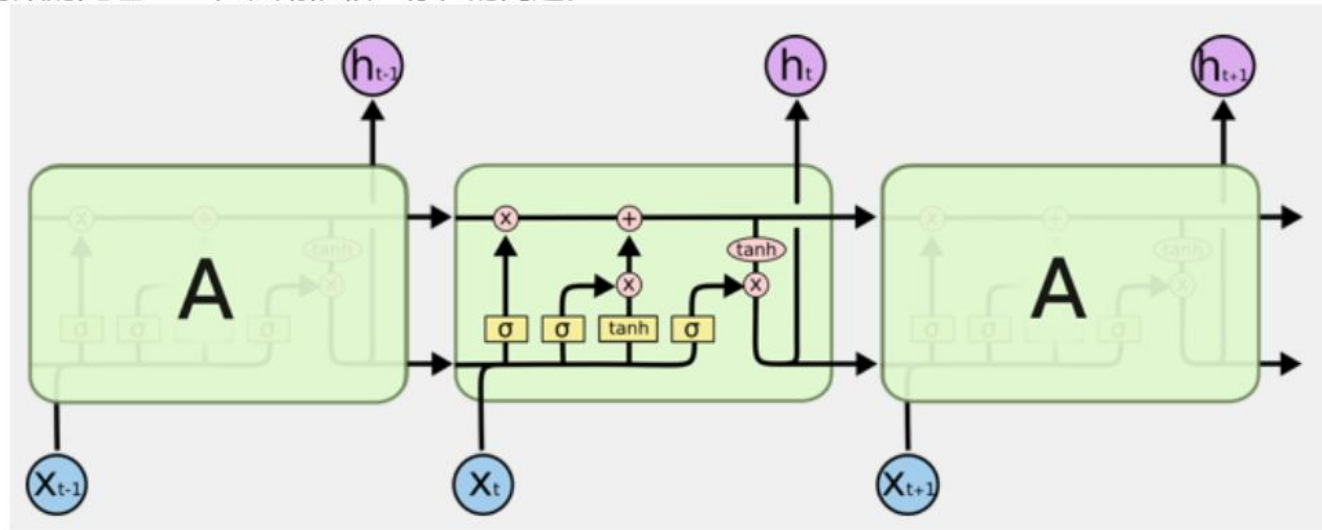
标准循环神经网络的另外一个问题在于能够存取的上下文信息范围很有限，隐含层的输入对于网络输出的影响随着时间的不断递归而衰退。



LSTM结构

- LSTM属于RNN的一种变种，对其网络结构进行了改进，具备避免梯度消失的特性，从而让RNN网络自身具备处理长期序列依赖的能力。
- LSTM网络对于梯度消失，采用特殊的方式存储“记忆”，以前梯度比较大的“记忆”不会像简单的RNN一样马上被抹除，可以一定程度上克服梯度消失问题。

长短期记忆网络（Long Short-Term Memory, LSTM）是一种特殊的RNN结构，通过对结构的设计可以有效的处理RNN在长期依赖上存在的问题。

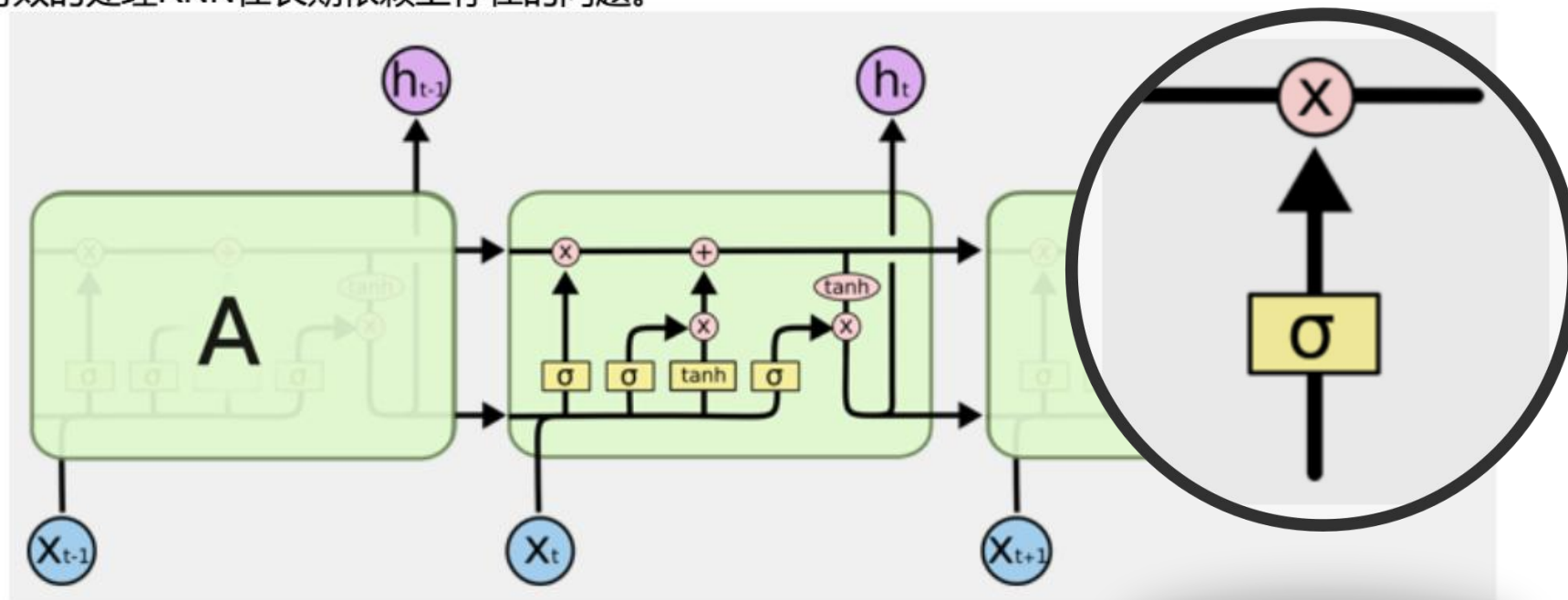


$$A = \pi r^2$$

LSTM (long short term memory)

- LSTM中最基本的就是“门” (gate) 的结构，让信息有选择性的影响网络中每个时刻的状态。“门”结构就是使用sigmoid神经网络和一个按照元素相乘的操作。

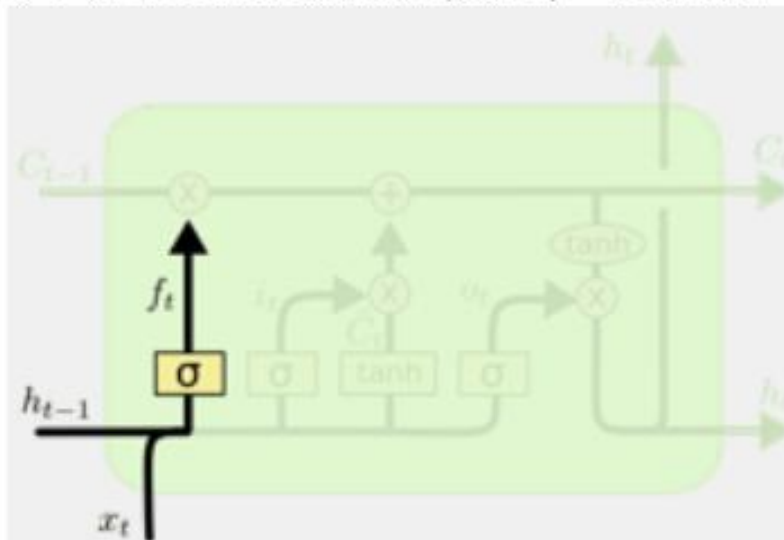
长短期记忆网络 (Long Short-Term Memory, LSTM) 是一种特殊的RNN结构，通过对结构的设计可以有效的处理RNN在长期依赖上存在的问题。



$$A = \pi r^2$$

遗忘门

“遗忘门”和“输入门”是LSTM结构的核心，“遗忘门”的作用是让LSTM“忘记”之前没有用的信息。

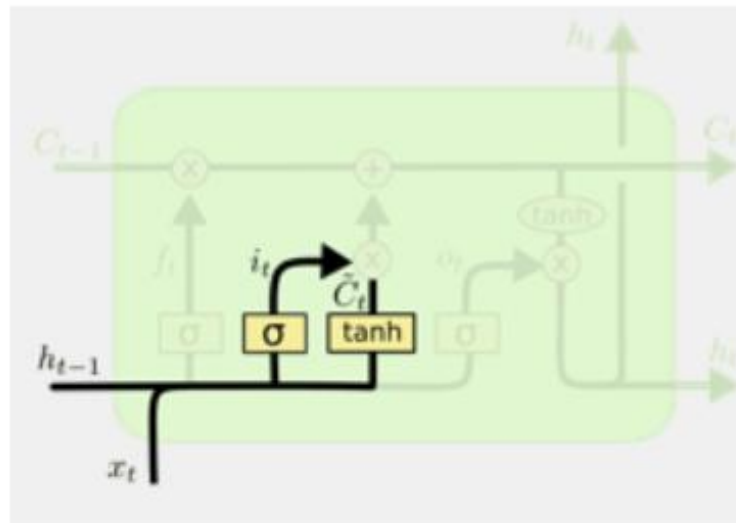


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- C_t - t 时刻的隐藏状态
- h_t - t 时刻的输出
- x_t - t 时刻的输入

输入门

LSTM忘记了之前的部分信息后还需要从当前的输入数据中补充新的数据信息，这样的一个过程就是“输入门”完成的。

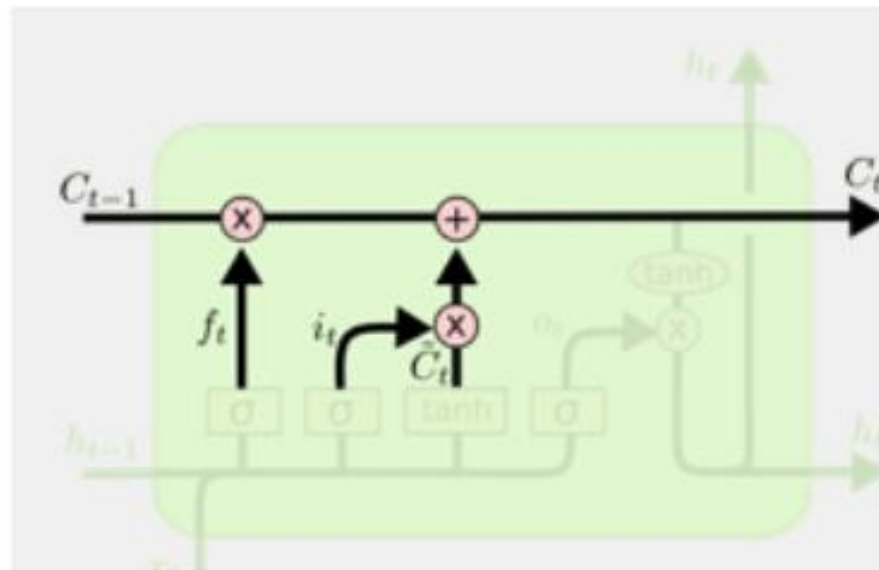


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- C_t - t 时刻的隐藏状态
- h_t - t 时刻的输出
- x_t - t 时刻的输入

更新状态

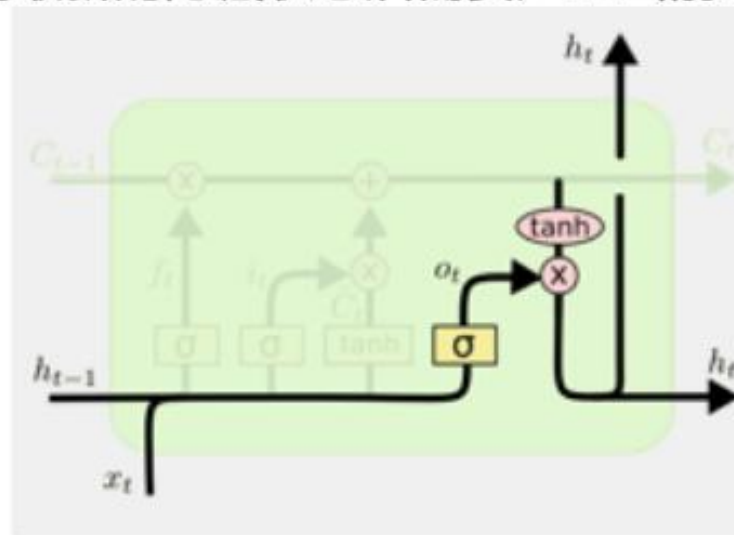


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- C_t - t 时刻的隐藏状态
- h_t - t 时刻的输出
- x_t - t 时刻的输入

输出门

LSTM在计算得到最新的隐藏状态后需要产生当前时刻的输出，这个过程是通过输出门来完成的。



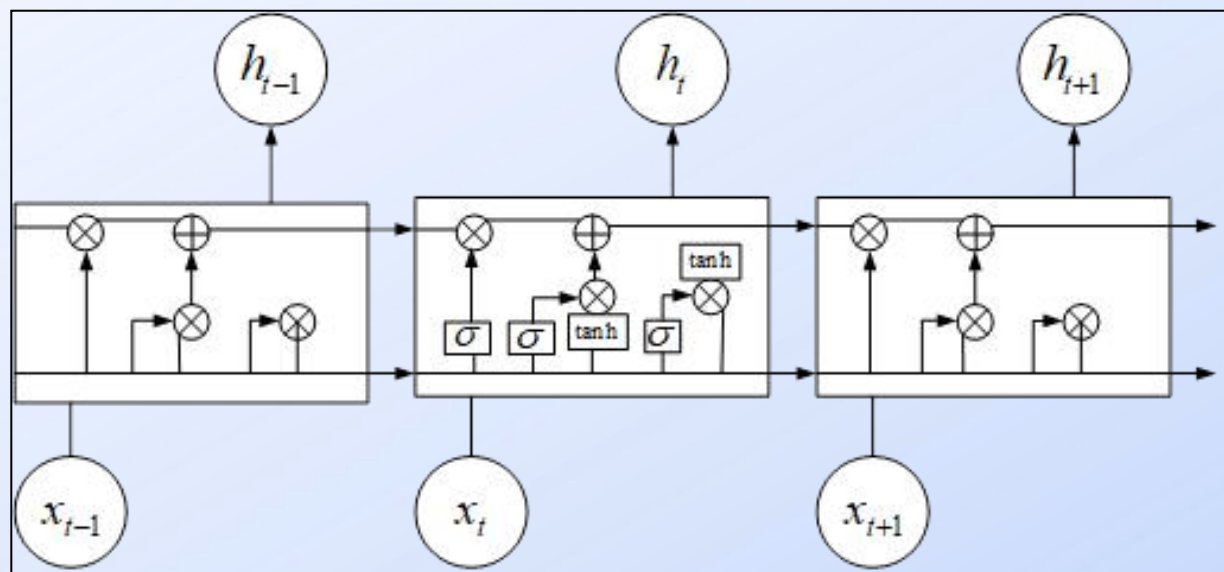
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

- C_t - t 时刻的隐藏状态
- h_t - t 时刻的输出
- x_t - t 时刻的输入

等长的多对多结构

- LSTM网络通过梯度剪裁技术克服梯度爆炸问题，当计算的梯度超过阈值 c 或者小于阈值 $-c$ 的时候，便将此时的梯度设置成 c 或 $-c$ 。
- 例如，3个神经网络的LSTM内部结构如图所示。



- 图中 $\times$ 表示乘法， $+$ 表示加法， $\tanh$ 表示 $\tanh$ 函数， σ 表示sigmoid函数，将数据压缩到0到1的范围，0表示信息无法通过该层，1表示信息可以全部通过。

$$A = \pi r^2$$



等长的多对多结构

- LSTM神经网络模型使用门结构实现了对序列数据中的遗忘与记忆，能够刻画出输入数据中的短时的相关信息，还能捕捉到具有较长时间间隔的依赖关系，能够很好地应用于文本数据的处理。
- 使用大量的文本序列数据对LSTM模型训练后，可以捕捉到文本间的依赖关系，训练好的模型就可以根据指定的文本生成后续的内容。



目录 CONTENTS



01

循环神经网络

02

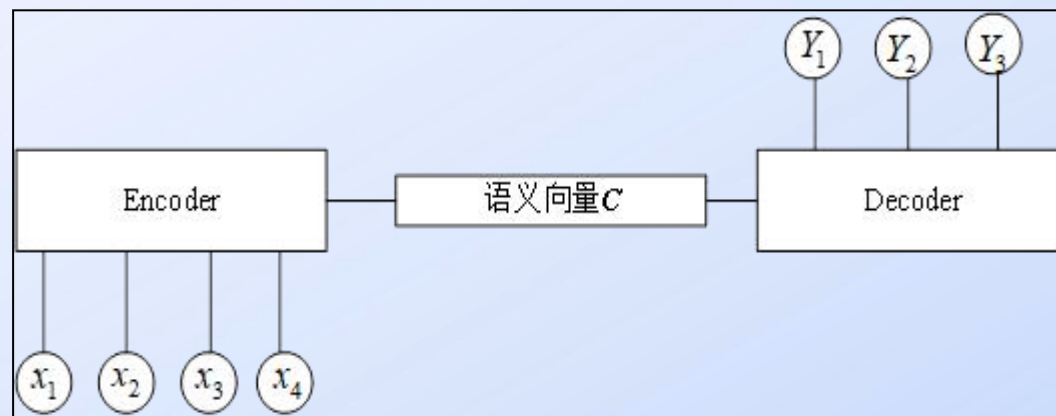
长短时记忆神经网络

03

Seq2Seq问答机器人

非等长结构 (Seq2Seq模型)

- Seq2Seq模型是由Google Brain和Yoshua Bengio两个团队在2014年各自独立提出来的模型结构，主要解决机器翻译问题。
- 最基础的Seq2Seq模型包含了3个部分，即编码器（Encoder）、解码器（Decoder）和连接两者的中间状态向量C，如图所示。



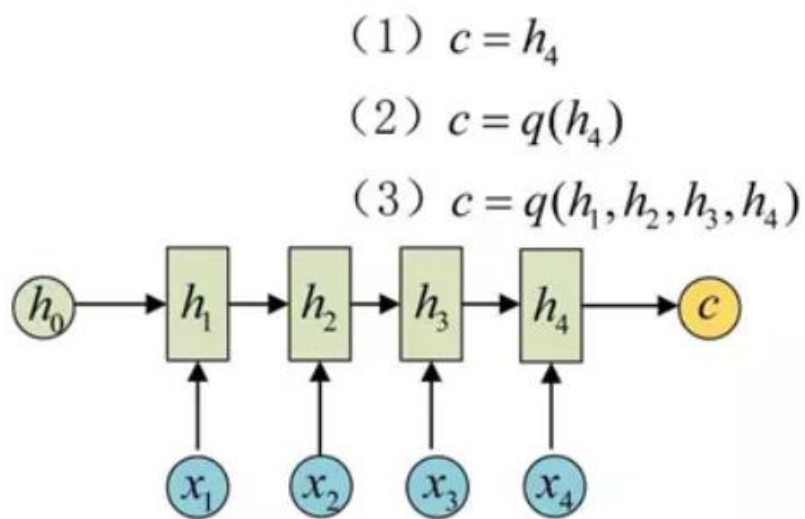
- Encoder通过学习输入，将其编码成一个固定大小的状态向量C，继而将C传给Decoder，Decoder再通过对向量C的学习进行输出。



Seq2Seq

- Seq2Seq模型是RNN最重要的一个变种：N vs M（输入与输出序列长度不同）。
- 这种结构又叫Encoder-Decoder模型。
- 原始的N vs N RNN要求序列等长，然而我们遇到的大部分问题序列都是不等长的，如机器翻译中，源语言和目标语言的句子往往并没有相同的长度。

为此，Encoder-Decoder结构先将输入数据编码成一个上下文向量 c ：

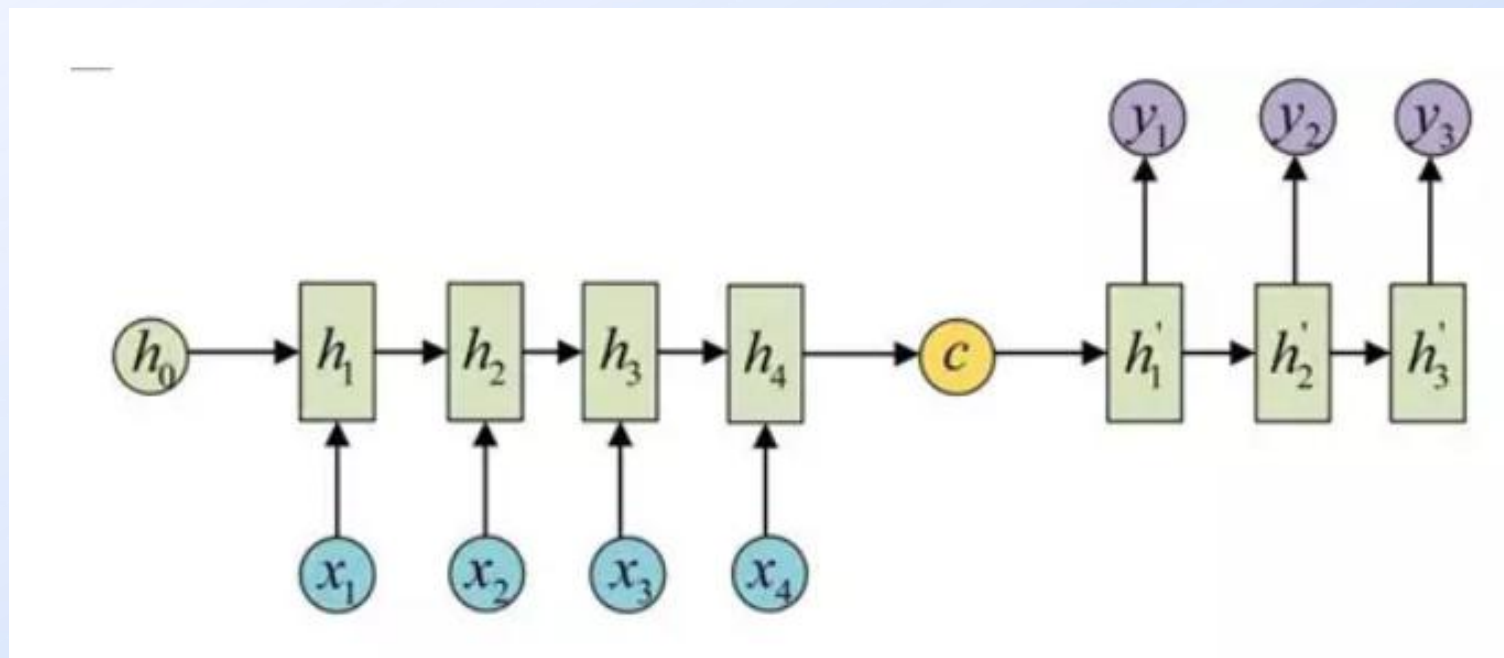


得到 c 有多种方式，最简单的方法就是把 $Encoder$ 的最后一个隐状态赋值给 c ，还可以对最后的隐状态做一个变换得到 c ，也可以对所有的隐状态做变换。

$$\frac{A}{r} = \pi r^2$$

Seq2Seq

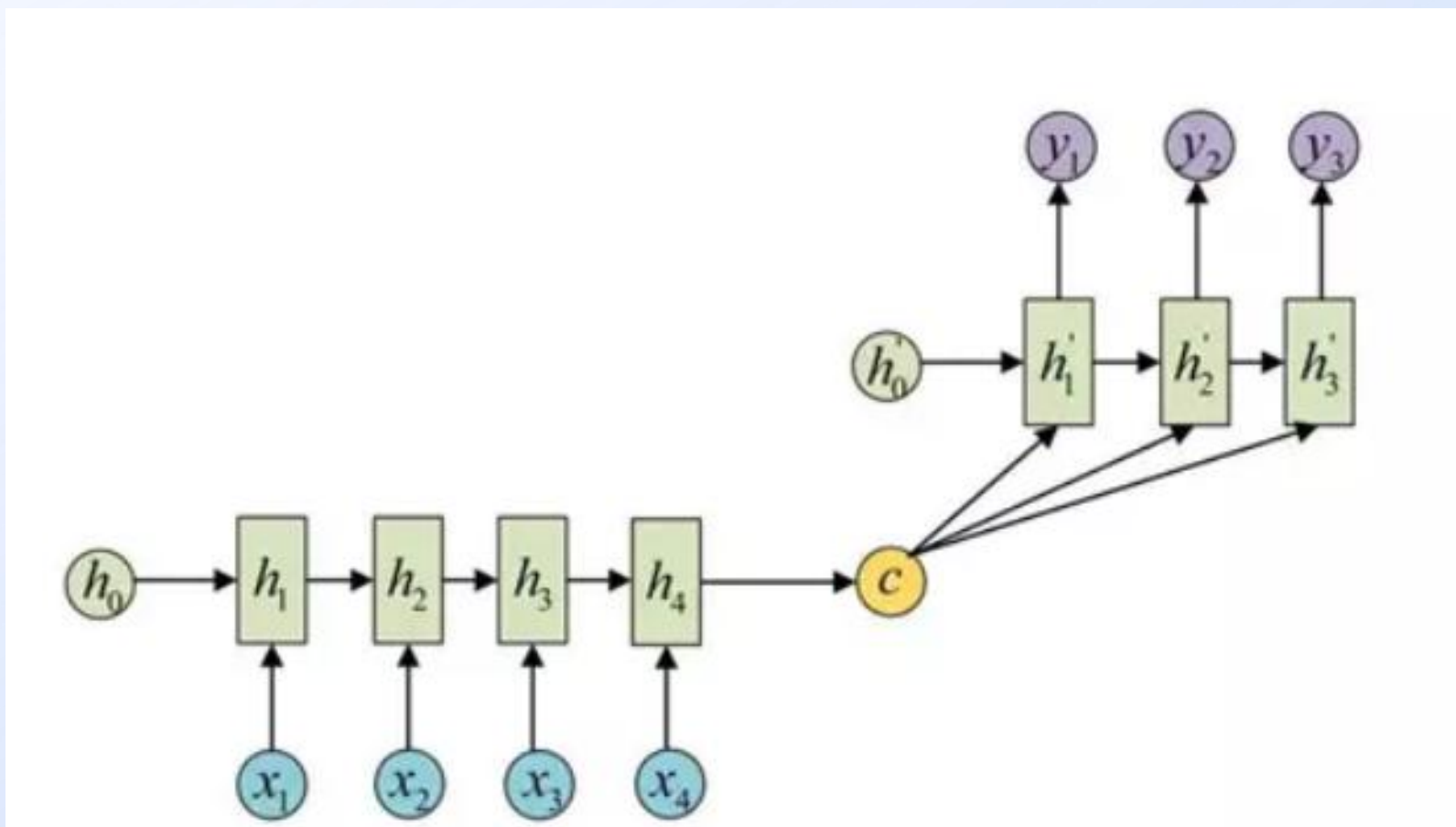
- 拿到 c 之后，就用另一个RNN网络对其进行解码，这部分RNN网络被称为Decoder。具体做法就是将 c 当做之前的初始状态 h_0 输入到Decoder中：



$$\frac{A}{r} = \pi r^2$$

Seq2Seq

- 还有一种做法是将 c 当做每一步的输入:



$$A = \pi r^2$$



Seq2Seq

- 由于这种Encoder-Decoder结构不限制输入和输出的序列长度，因此应用的范围非常广泛，比如：
- 机器翻译。Encoder-Decoder的最经典应用，事实上这一结构就是在机器翻译领域最先提出的
- 文本摘要。输入是一段文本序列，输出是这段文本序列的摘要序列。
- 阅读理解。将输入的文章和问题分别编码，再对其进行解码得到问题的答案。
- 语音识别。输入是语音信号序列，输出是文字序列。





非等长结构 (Seq2Seq模型)

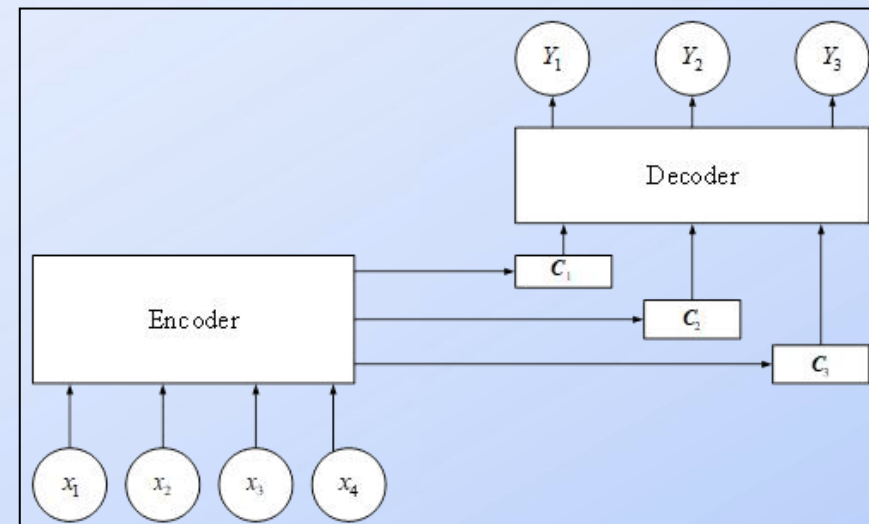
- 由于Encoder-Decoder结构没有输入和输出等长的限制，因此应用的范围非常广泛。有以下四方面。
 - 机器翻译：Encoder-Decoder的经典应用之一，事实上这一结构就是在机器翻译领域最先提出的。
 - 文本摘要：输入是一段文本序列，输出是这段文本序列的摘要序列。
 - 阅读理解：将输入的文章和问题分别编码，再对其进行解码得到问题的答案。
 - 语音识别：输入是语音信号序列，输出是文字序列。



非等长结构 (Seq2Seq模型)

• 1. Attention机制

- 在Encoder-Decoder结构中，Encoder将所有的输入序列都编码成一个统一的语义特征向量C再解码。因此，向量C中必须包含原始序列中的所有信息，它的长度就成了限制模型性能的瓶颈。如在机器翻译问题中，当翻译的句子较长时，一个向量C可能存不下那么多信息，就会造成翻译精度的下降。
- 注意力（Attention）机制通过在解码（Decoder）过程中每个节点输入不同的向量C解决这个问题，带有Attention机制的Decoder如图所示。





非等长结构 (Seq2Seq模型)

- 2. Seq2Seq模型的目标函数
- 用 $x = \{x_1, x_2, \dots, x_n\}$ 代表输入的语句, $y = \{y_1, y_2, \dots, y_n\}$ 代表输出的语句, y_t 代表当前输出词, 则Seq2Seq模型的目标函数为: $P(y|x) = \prod_{t=1}^m P(y_t|y_1, y_2, \dots, y_{t-1}, x)$ 。输出的 y_t 不仅依赖之前的输出 $\{y_1, y_2, \dots, y_{t-1}\}$, 还依赖输入语句 x 。
- 上式中存在数值下溢问题。原因是式中每一项 $P(y_t|y_1, y_2, \dots, y_{t-1}, x)$ 都小于1甚至远小于1, 很多项乘起来会得到很小的数字, 造成数值下溢。因此, 在实际中一般是取log值, 求其概率的对数和而不是概率的乘积。此时Seq2Seq模型最大化目标函数为: $P(y|x) = \sum_{t=1}^m \log P(y_t|y_1, y_2, \dots, y_{t-1}, x)$ 。





目录

1	循环神经网络概述
2	RNN结构
3	深度学习工具
4	基于LSTM的文本分类与情感分析
5	基于Seq2Seq的机器翻译





深度学习工具

- 在深度学习初始阶段，每个深度学习研究者都需要编写大量的重复代码。
- 为了提高工作效率，研究者们就将这些代码写成了一个框架放到互联网供研究者使用。
- 目前，最为流行的深度学习框架有Paddle、TensorFlow、Caffe、Theano、MXNet、Torch和PyTorch。
- 深度学习框架提供了一系列的深度学习的组件，降低了入门的门槛，不需要从复杂的神经网络开始编代码。
- 可以根据需要选择已有的模型，通过训练得到模型参数；也可以选择自己需要的分类器和优化算法，然后调用深度学习框架的函数接口使用用户自定义的新算法。





TensorFlow简介

- TensorFlow是一个开源软件库，使用数据流图进行数值计算，是谷歌公司研发的第二代人工智能学习系统。
- **Tensor（张量）表示着N维数组**，Flow（流）表示基于数据流图的计算，TensorFlow为张量从数据流图的一端流动到另一端的计算过程。
- TensorFlow将复杂的数据结构传输至人工智能神经网络进行分析和处理。





Tensorflow特点

- **灵活性**：TensorFlow是一个“神经网络”库，用户可以自己用Python描绘计算图，而后放到计算核心之中。
- **可移植性**：TensorFlow可以在CPU、GPU上运行，如台式机、服务器、集群和移动设备上。训练好的模型也可以存放到移动设备上。
- **综合了科研以及产品**：以往的机器学习算法的研究中往往需要自己编写大量代码，而TensorFlow的可以通过其简单的验证想法，并将其直接输出产品。
- **自动计算梯度导数**：TensorFlow可以自动计算函数导数，只需关于模型的定义与验证。
- **性能最优化**：TensorFlow底层上对于线程、队列、异步操作给予了良好的支持，使得其可以很好的发挥出硬件的全部性能。而在多计算单元控制上，可以将不同的计算任务分配到不同的单元之中。
- **多语言支持**：TensorFlow支持C++、Python、Java、Go、JavaScript API。





Windows环境下TensorFlow的安装

Windows环境下安装TensorFlow的步骤有以下五步。

- 在开始菜单中找到Anaconda Prompt，单击运行。
- 将TensorFlow源更换为国内镜像（安装速度更快），镜像选择可以是清华、豆瓣镜像等。
- 检查新环境中的Python版本，通过version参数实现。
- 安装Python版本对应的TensorFlow版本。
- 安装完成后，验证TensorFlow是否安装成功。先在终端输入命令“python”，进入Python交互界面，然后输入导入命令，若没有报错信息，则说明TensorFlow安装成功。





基于TensorFlow的深度学习库Keras

- Keras是一个高层神经网络API，是一个上层封装的工具库，基于TensorFlow、Theano、MXNet以及CNTK框架。
- Keras上手简单，适合应用于小型环境（实验室、数据竞赛）。
- Keras具有简易和快速的原型设计，支持CNN与RNN，能够无缝在CPU与GPU间切换的优点。
- 为了避免因版本升级带来的一些功能函数的变化的修改，可以选择安装指定版本的Keras。





目录

1	循环神经网络概述
2	RNN结构
3	深度学习工具
4	基于LSTM的文本分类与情感分析
5	基于Seq2Seq的机器翻译





文本分类

文本分类模型采用两层LSTM网络，训练数据集为THUCNews的一个子集。文本分类的流程，主要包括以下4个步骤。

- **语料预处理：**读取THUCNews数据，并进行语料预处理的工作，包括统计词频、生成字库、根据字库将每条文字转化为一个向量。
- **模型构建：**定义LSTM网络框架，设置相关参数后查看模型结构。
- **模型训练：**对模型进行训练并保存模型。
- **模型测试：**使用训练好的模型对验证集进行测试和评价。





文本分类-Step1 语料预处理

- 训练集使用THUCNews数据中10个分类，包括体育、财经、房产、家居、教育、科技、时尚、时政、游戏和娱乐。其中每个分类包含6500条数据，训练集5000条数据，验证集500条数据，测试集1000条数据。
- 语料预处理cnews_loader.py定义7个函数，其内容如下。
 - open_file打开文件，read_file读取文件。
 - build_vocab构建词汇表。
 - read_vocab读取上一步存储的词汇表，并转换为{词: id}表示。
 - read_category将分类目录固定，转换为{类别: id}表示。
 - to_words将id表示的数据转换为文字，process_file将数据集从文字转换为固定长度的id序列表示。
- 接着使用定义好的函数加载训练数据、验证数据、测试数据并分别进行预处理。





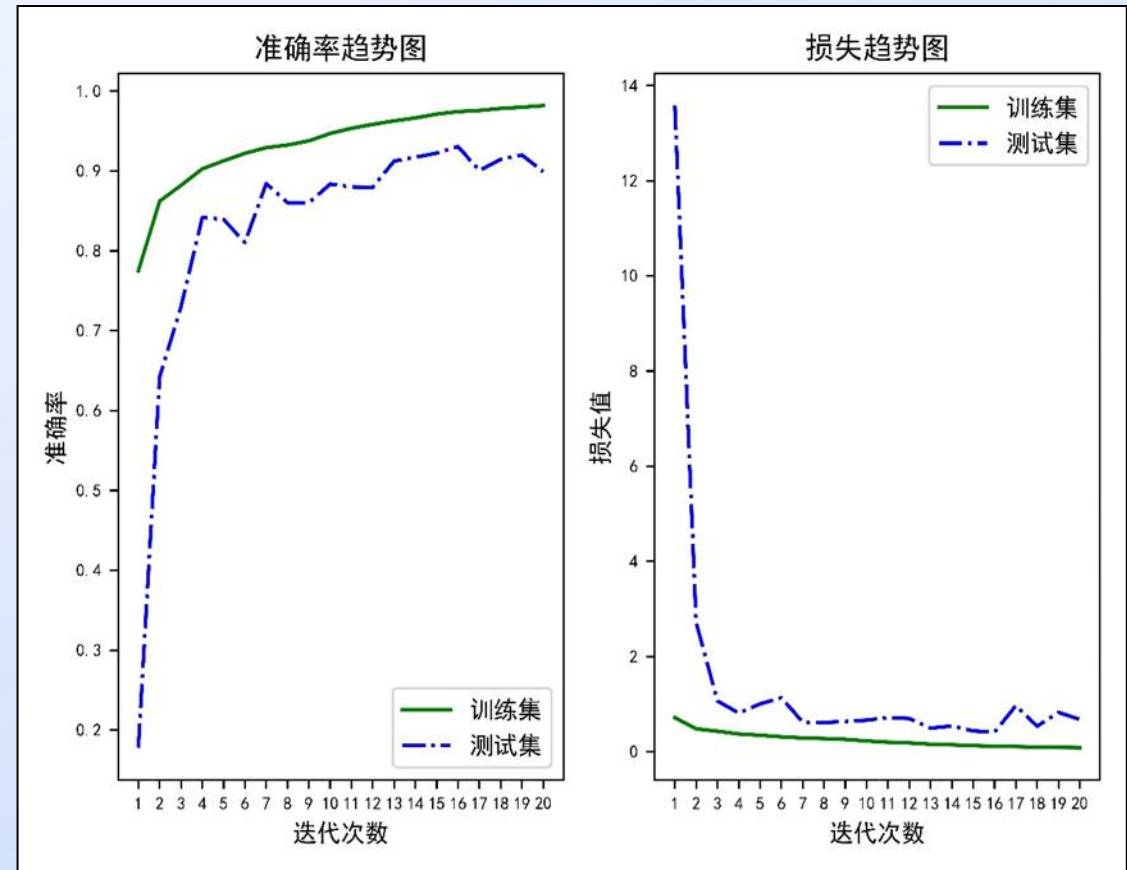
文本分类-Step2 构建模型

- 对LSTM模型的参数进行设置，设置的内容如下。
 - 设置词向量维度为128，输入词序列长度为600，类别数为10，词汇表大小为5000。
 - 隐藏层层数为2，隐藏层神经元个数分别为256和128。
 - 学习率为0.001，每批训练数据个数为64，总迭代轮次20。
- 设置好模型参数后构建模型。



文本分类-Step3 模型训练

- 对构建好的模型进行训练参数的设置。设置损失值指标为categorical_crossentropy, 优化器为RMSprop, 评价指标为categorical_accuracy。设置训练次数为20次, 然后进行模型的训练。
- 模型训练得到的结果如图所示。
- 从结果可以看出, 随着迭代次数的上升, 模型在训练集上的准确率逐渐上升, 损失值逐渐下降, 模型预测效果越来越精确。最后通过save函数保存模型。





文本分类-Step 4 模型测试

- 导入load_model模块用于读取本地模型，之后调用训练好的模型进行测试。
- 通过sklearn库下的metrics模型评价方法，对模型的预测结果进行评价，结果如图所示。
- 由模型评价结果可以看出，模型在测试集上的准确率（accuracy）达到了0.96，且各类的精确率（precision）、召回率（recall）和F1值（f1-score），除了家居（编号为3）的精确率，其他都超过了0.9。

	precision	recall	f1-score	support
0	0.99	1.00	0.99	990
1	0.96	0.96	0.96	1000
2	1.00	1.00	1.00	997
3	0.86	0.97	0.91	887
4	0.93	0.94	0.94	984
5	0.97	0.94	0.96	1030
6	0.97	0.92	0.95	1047
7	0.94	0.93	0.94	1010
8	0.98	0.95	0.96	1034
9	0.98	0.96	0.97	1021
accuracy			0.96	10000
macro avg	0.96	0.96	0.96	10000
weighted avg	0.96	0.96	0.96	10000

文本分类

- 除模型评价结果外，还可得到混淆矩阵如图所示。
- 由混淆矩阵结果图可以看出，模型在标签的预测上除了家居外，其余精确率均达到了90%以上，模型效果较好。

预测标签	体育	990	0	0	0	2	2	0	0	3	3
	财经	0	965	1	2	5	3	3	19	1	1
	房产	0	0	996	3	1	0	0	0	0	0
	家居	0	16	0	856	17	15	53	32	6	5
	教育	0	3	0	7	929	12	8	11	25	5
	科技	0	2	0	3	1	972	5	4	12	1
	时尚	0	0	0	7	6	1	968	1	1	16
	时政	0	11	0	6	15	18	0	941	2	7
	游戏	0	1	0	0	2	3	6	1	980	7
	娱乐	0	2	0	3	6	4	4	1	4	976
		体育	财经	房产	家居	教育	科技	时尚	时政	游戏	娱乐
真实标签											



情感分析

数据来源为某电商平台的热水器评论数据，其中正面评论10677条，负面评论10428条。本案例的目的是通过构建模型识别各条评论的情感倾向，即情感分析，正面评论为1，负面评论为0。

情感分析的步骤有以下五步。

- 读取正负情感语料。
- 评论词语向量化。
- 模型构建。设置LSTM模型参数，嵌入层输入的维度为字典的长度加1，输出维度为256，输入词的序列长度为50，类别数为2，隐藏层层数为1，隐藏层神经元个数为128。
- 模型训练。每批训练数据个数为16，总迭代轮次为10。
- 模型预测和评价。





情感分析-Step 1读取正负情感语料

读取正负情感的语料库，由于情感分析的目的是将数据划分为正面和负面标签，故还需将正负语料的数据进行“贴标签”处理，正面评论标签为1，负面评论标签为0。





情感分析-Step2评论词语向量化

- 由于数据为评论文本的形式，不能直接用于建模，因此要先对评论数据进行文本的分词处理，计算每个词语出现的频次，将分词结果进行向量化，变为模型可以识别的数据形式。
- 每一串索引的长度并不相等，所以为了方便模型的训练，需要将索引的长度标准化，这里每条评论取50个词作为标准，通过sklearn库的sequence方法进行标准化。
- 将数据划分为训练集和测试集以便建模训练，最后是将评论文本中的词语向量化。





情感分析-Step 3模型构建

- 设置LSTM模型参数，嵌入层输入的维度为字典的长度加1，输出维度为256，输入词的序列长度为50，类别数为2，隐藏层层数为1，隐藏层神经元个数为128。
- 设置好模型参数并构建LSTM模型。





情感分析-Step 4模型训练

- 搭建好的模型，通过compile设置损失值为“binary_crossentropy”，优化器为“Adam”，评价指标为“accuracy”，每批训练数据个数为16，总迭代轮次为10，设置好模型超参并进行训练。
- 运行代码后得到模型在测试集上的准确率（accuracy）达到了99.77%，损失值（loss）随着模型的训练不断下降。





情感分析-Step 5 模型测试

- 对训练好的模型使用测试数据的进行测试，通过sklearn库的模型评价，输出模型的评价结果如图所示。
- 由结果可以看出，模型在测试集上的准确率（accuracy）达到了0.89，且各类的精确率（precision）、召回率（recall）和F1值（f1-score）都超过了0.89，认为情感分析模型的效果较好。之后可以通过调整模型的参数以达到更好的效果。

测试集的准确率为：0.8921735834754595

精确率，召回率，F1 值分别为：

	precision	recall	f1-score	support
0	0.90	0.88	0.89	2604
1	0.89	0.90	0.89	2673
avg / total	0.89	0.89	0.89	5277

混淆矩阵为：

[[2296 308]

[261 2412]]





目录

1	循环神经网络概述
2	RNN结构
3	深度学习工具
4	基于LSTM的文本分类与情感分析
5	基于Seq2Seq的机器翻译





基于Seq2Seq的机器翻译

- 采用Seq2Seq模型，借助GPU构建中英文机器翻译模型。模型构建和训练的环境配置有如下四点。
 - Linux操作系统为ubuntu 16.04。
 - GPU为Nvidia GTX 1080Ti。
 - Python 3.8.3。
 - TensorFlow 2.3。
- 机器翻译的流程包括以下3个步骤。
 - **语料预处理**：读取原始数据并解析文件，分别对中英文内容进行分词，筛选规定词数的句子，保存至新文件中。
 - **训练模型**：对新数据划分训练集和测试集，设置相关参数后对模型进行训练与验证，保存模型。
 - **模型测试**：读取训练好的模型，对模型进行测试。





语料预处理

原始数据是以txt的格式存储数据，包含部分中英文句子互译的内容。语料预处理过程包括以下四步。

- 利用io对原始数据文本内容进行读取，并在每个句子中添加一个开始和结束的标记。
- 由于语料数据太多，从语料库选用一部分数据进行数据处理和建模即可。选用数据后，通过删除特殊字符清理句子，并返回格式为[英语，中文]的单词对。
- 创建一个单词索引和一个反向单词索引（从单词→id和id→word到字典的映射），构造英语与中文的映射，即将单词转换为数字的字典；构造英语与中文的反向映射，即从数字转换为单词的字典。
- 将每个句子填充到最大长度，并创建一个tf.data数据集，以提升运算速度。





构建模型

- 构建模型包括以下3个步骤。
 - 定义编码器的输入，定义LSTM层，调用编码器，得到编码器的输出以及状态信息，定义解码器的输入，将编码器输出的状态作为初始解码器的初始状态，添加全连接层，定义整个模型。
 - 添加注意力机制，原始编解码模型的encode过程会生成一个中间向量C，用于保存原序列的语义信息。但是这个向量长度是固定的，当输入原序列的长度比较长时，向量C无法保存全部的语义信息，所以使用Attention机制打破这种原始编解码模型对固定向量的限制。
 - 模型中相关参数的配置，设置训练周期为10次，每批数据量大小为256，每次随机抽取256条内容。
- 构建基于Seq2Seq的机器翻译模型，定义三个类。其中，Encoder类编写编码器，BahdanauAttention类添加注意力机制，Decoder类编写解码器。





定义优化器及损失函数

定义优化器及损失函数包括以下3个步骤。

- 选择Adam优化器，它是一种可以替代传统随机梯度下降过程的一阶优化算法，它能基于训练数据迭代地更新神经网络权重。
- 选择SparseCategoricalCrossentropy为损失值指标，它是指用于计算标签值和预测值之间差异的函数，可以最小化每个训练样例的误差。
- loss_function函数实现优化器及损失函数。





训练模型

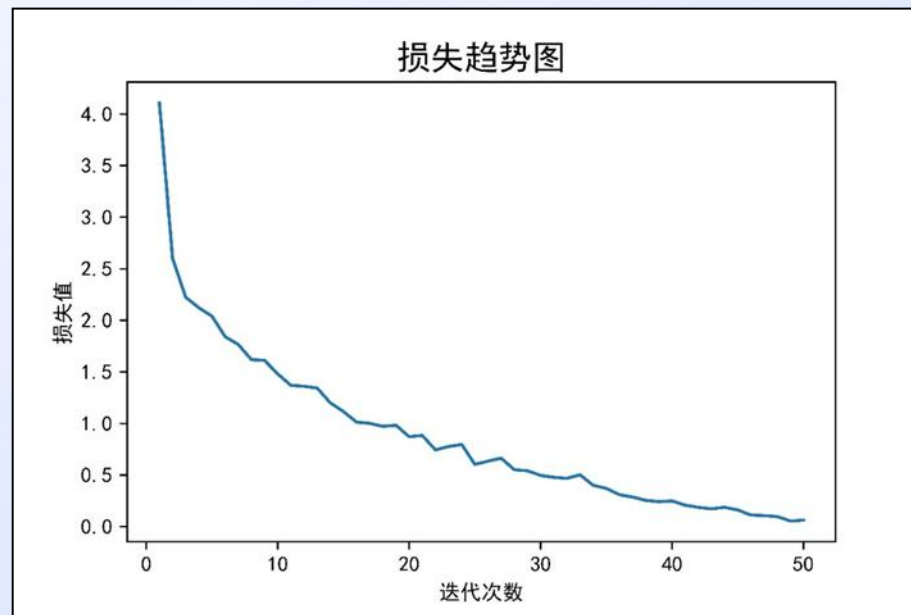
训练模型包括以下5个步骤。

- 将数据集输入通过编码器，编码器返回编码器输出和编码器隐藏状态。
- 编码器输出，编码器隐藏状态和解码器输入（即开始令牌）被传递到解码器。
- 解码器返回预测，并且解码器的隐藏状态，然后将解码器隐藏状态传递回模型，并使用预测计算损耗。
- 利用注意力机制决定解码器的下一个输入，其中注意力机制是将目标单词作为下一个输入传递给解码器的技术。
- 计算梯度并将其应用于优化器和反向传播。



训练模型

- 构建好训练模型后，使用train函数实现训练及验证模型，设置训练次数为50次。运行代码，得到损失趋势折线图如图所示。



- 由结果图可以看出，模型在训练集上损失值（Loss）随着迭代次数（epochs）增加而缓慢下降，并趋向于稳定，误差值越来越小，则模型预测的会越来越精确。

$$\frac{A}{r} = \pi r^2$$



翻译

- 翻译主要包括以下两项内容。
 - 与训练模型时类似，设置测试时的相关参数配置。
 - 输入五条句子，利用模型进行翻译，评价模型翻译的准确性。
- 使用训练好的模型对语句进行翻译，其中evaluate函数实现模型测试，translate函数实现语句的翻译。





小结

- 本章主要介绍NLP中使用的深度学习技术，包括循环神经网络、长短期记忆网络和Seq2Seq模型。
- 通过介绍循环神经网络的基本概念，引入单向RNN结构、LSTM模型结构以及Seq2Seq模型结构。
- 介绍常见的深度学习工具。
- 最后讲述LSTM模型在文本分类和情感分析的应用，Seq2Seq模型在机器翻译的应用。



THANK YOU

